

Appendix

1 Optimality Gap Analysis for MigOpt

MigOpt approximates Eq. (1) by routing every migration through T_{N-1} (§3.2). This relaxation makes the problem tractable, but inflates the per-migration cost relative to the true optimum and may bias MigOpt toward fewer migrations. We claimed in §3.2 that the resulting optimality gap is 5.7%. This section substantiates that claim.

The true optimum, denoted z_{opt} , is unknown because Eq. (1) is NP-hard (§3.1). We instead derive a computable lower bound z_{LB} satisfying $z_{\text{LB}} \leq z_{\text{opt}}$, and report $(z_{\text{MigOpt}} - z_{\text{LB}})/z_{\text{LB}}$ as an upper bound on the true gap. The remainder of this section reformulates Eq. (1) as an integer linear program (§1.1), relaxes the integrality constraint to obtain a linear program whose optimum lower-bounds the integer optimum (§1.2), and reports the gap on the evaluated workloads (§1.3).

1.1 Reformulating Objective Function as an Integer Linear Program

Before investigating the optimality gap, we first build an integer linear program (ILP) for the migration problem. For each page $p \in P$, time step $t \in \{1, \dots, E\}$, and tier pair $(i, j) \in \{0, \dots, N-1\}^2$, let $y_{p,t,i,j} \in \{0, 1\}$ indicate whether page p transitions from T_i at time $t-1$ to T_j at time t . The remainder of this subsection expresses every quantity used in Eq. (1) as a linear function of y , and then collects them into the ILP. Eq. (1) in §3.1 was presented as:

$$\sum_{t=0}^{E-1} \sum_{p \in a_t} A(T_{l(p,t)}) + \sum_{t=1}^E \sum_{p \in M_t} [R(T_{l(p,t-1)}) + W(T_{l(p,t)})].$$

To build a linear function, we first express $l(p, t)$, the placement variable (i.e., the location of the page p at time t) using $y_{p,t,i,j}$. By definition, $y_{p,t,i,j} = 1$ means page p ends up in T_j at time t , regardless of the source T_i . Because each page makes exactly one transition at every time step, exactly one variable $y_{p,t,i,j}$ over (i, j) takes the value one for each (p, t) (i.e., at time t , no page exists in multiple tiers simultaneously). Therefore, the sum of $y_{p,t,i,j}$ over $i = 0$ to $N-1$ satisfies:

$$l(p, t) = j \iff \sum_{i=0}^{N-1} y_{p,t,i,j} = 1 \quad \forall p, \forall t \in \{1, \dots, E\},$$

which means for all T_i , there is only one case where a specific page goes to T_j .

The next step is to write the access latency $A(T_{l(p,t)})$ in y . $A(T_{l(p,t)})$ is simply the latency of the tier that holds page p at time t . Since p sits in exactly one tier, we can list the latencies of all N tiers and pick out the one that matches:

$$A(T_{l(p,t)}) = \sum_{j=0}^{N-1} A(T_j) \cdot \mathbb{I}[l(p, t) = j],$$

where $\mathbb{I}[\cdot]$ equals one if the condition holds and zero otherwise. Here $\mathbb{I}[l(p, t) = j]$ equals one for the single tier j that holds p and zero for every other tier, so only one term in the sum survives. We have already shown that $\sum_i y_{p,t,i,j}$ is exactly this indicator, so substituting gives

$$A(T_{l(p,t)}) = \sum_{j=0}^{N-1} A(T_j) \sum_{i=0}^{N-1} y_{p,t,i,j}. \quad (1)$$

The right-hand side is linear in y : each variable $y_{p,t,i,j}$ enters with a fixed coefficient $A(T_j)$.

We apply the same idea to the migration cost. Its building blocks are $R(T_{l(p,t-1)})$ and $W(T_{l(p,t)})$: the read at the source tier and the write at the destination tier of one transition.

$W(T_{l(p,t)})$ has the same form as $A(T_{l(p,t)})$ in the previous paragraph: both depend on the destination tier of p at time t . Reusing the indicator $\sum_i y_{p,t,i,j} = \mathbb{I}[l(p, t) = j]$ gives

$$W(T_{l(p,t)}) = \sum_{j=0}^{N-1} W(T_j) \sum_{i=0}^{N-1} y_{p,t,i,j}.$$

$R(T_{l(p,t-1)})$, in contrast, depends on the source tier at time $t-1$. To detect this source, we sum $y_{p,t,i,j}$ over the destination j instead of the source i . If p sits in T_i at time $t-1$, exactly one variable $y_{p,t,i,j}$ over j is one (the one whose destination matches p 's tier at time t), so

$$\sum_{j=0}^{N-1} y_{p,t,i,j} = \mathbb{I}[l(p, t-1) = i].$$

Substituting yields

$$R(T_{l(p,t-1)}) = \sum_{i=0}^{N-1} R(T_i) \sum_{j=0}^{N-1} y_{p,t,i,j}.$$

Adding the two terms, the per-migration cost rewrites as a single double sum over (i, j) :

$$R(T_{l(p,t-1)}) + W(T_{l(p,t)}) = \sum_{i,j} [R(T_i) + W(T_j)] y_{p,t,i,j}, \quad (2)$$

in which each variable $y_{p,t,i,j}$ enters with the fixed coefficient $R(T_i) + W(T_j)$.

With Eq. 1 and Eq. 2 fully written in y , the only remaining piece is the set of constraints that make y correspond to a valid placement schedule, shown in Table 1. Without them, y could place a page in two tiers at once (constrained by flow conservation (C1)), exceed a tier's capacity (constrained by tier capacity (C2)), or migrate more pages per step than the system allows (constrained by migration quota (C3)). We attach three constraints to y , corresponding to each of these requirements: flow conservation, tier capacity, and migration quota.

By combining the rewrites above with the three constraints, we obtain an ILP whose solution yields the optimal migration schedule $\{l(p, t)\}_{p,t}$ that minimizes Eq. (1), and the result is z_{opt} . However, the ILP is also NP-hard. We therefore turn

Table 1. Constraints for the ILP formulation

	Expression	Meaning
C1	$\sum_t y_{p,t,i,j} = \sum_k y_{p,t+1,j,k} \quad \forall p, j, t < E$	Flow conservation
C2	$\sum_i y_{p,t,i,j} \leq C(T_j) \quad \forall t, j$	Tier capacity
C3	$\sum_{p,i} y_{p,t,i,j} \leq Q \quad \forall t$	Migration quota

to the linear relaxation in §1.2 to obtain a tractable lower bound on z_{opt} .

1.2 Linear Programming Relaxation

A tractable lower bound on z_{opt} follows from replacing the integrality requirement with the box constraint

$$0 \leq y_{p,t,i,j} \leq 1.$$

All other constraints and the objective remain unchanged. We denote the optimal value of the resulting linear program (LP) by z_{LB} .

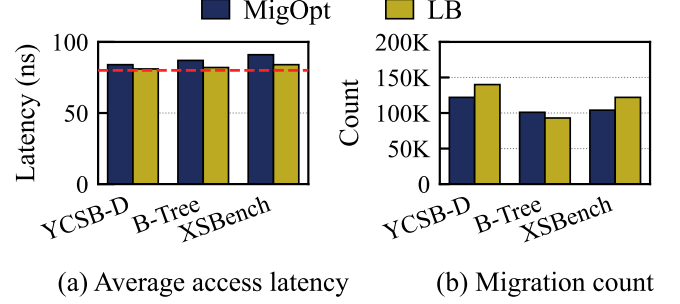
The fractional solution of the LP is physically meaningless. A page may, for instance, occupy T_0 with weight 0.3 and T_1 with weight 0.7 at the same time, which corresponds to no realizable schedule. The cost z_{LB} itself, however, is a rigorous lower bound on every integer-feasible schedule, by a two-step argument. First, every integer-feasible assignment $y \in \{0, 1\}$ satisfies $y \in [0, 1]$, so the LP feasible region contains every ILP-feasible point. Second, the LP and ILP share the same objective. Minimizing the same objective over a larger feasible region yields a value no greater than the integer minimum, so

$$z_{\text{LB}} \leq z_{\text{opt}} \leq z_{\text{MigOpt}}.$$

The reported gap $(z_{\text{MigOpt}} - z_{\text{LB}})/z_{\text{LB}}$ is therefore an upper bound on the true gap.

The LP is solvable in polynomial time by interior-point methods [2]. We solve it with the implementation in HiGHS [1] via its Python bindings. However, the LP has $|P| \cdot E \cdot N^2$ variables, which is too large at full scale; the XSBench trace alone has 91,812 pages. We shrink the problem by noticing that the LP cannot tell apart two pages with the same initial tier and the same access pattern over time. Such pages enter the LP with identical coefficients in the objective, the capacity constraints, and the quota constraints, so swapping their variables in any feasible solution gives another feasible solution of the same cost. The LP has no reason to treat them differently.

We therefore group pages that share these two characteristics into equivalence classes. Each class c is represented in the LP by a single variable set $y_{c,t,i,j}$, and its contribution is weighted by the class size m_c , the number of pages the class stands for. The capacity constraint becomes $\sum_c m_c \sum_i y_{c,t,i,j} \leq C(T_j)$ rather than $\sum_p \sum_i y_{p,t,i,j} \leq C(T_j)$, since placing a class

**Fig. 1.** Optimality gap analysis for MigOpt

into T_j occupies m_c slots at once instead of one. The quota constraint scales the same way. The aggregated LP has exactly the same optimum as the original, which we verify against the full LP on a prefix of each trace. On XSBench, aggregation reduces 91,812 pages to 29,387 classes.

The full XSBench trace contains 9.3M accesses, which corresponds to $E = 465$ when each interval covers the 20,000 memory instructions. At this E , the LP has roughly 2×10^8 variables, which is too large to solve directly. We therefore sweep $E \in \{1, 2, \dots, 50\}$ and fit a regression. The value $z_{\text{LB}}(E)$ is monotonically non-increasing in E , since every schedule feasible at coarse E remains feasible when each interval is subdivided. We fit a power-law regression

$$z_{\text{LB}}(E) \approx z_{\infty} + C \cdot E^{-\alpha}$$

to the measured values and adopt z_{∞} as the regression estimate. On all evaluated workloads, the fit residual is below 0.5% of z_{LB} , and $z_{\text{LB}}(E)$ flattens by $E = 8$.

1.3 Comparison with MigOpt

We compare z_{MigOpt} to the regression-based z_{LB} on the three workloads of §4: YCSB-D, B-Tree, and XSBench, where z_{MigOpt} is the total cost of the schedule produced by MigOpt. The tier configuration is shown in Table 2, and the per-step quota is set to $Q = 200$. Fig. 1 reports the resulting gap on three metrics: total cost, induced access cost, and induced migration count.

MigOpt shows the optimality gap as 5.7%, on average, of z_{LB} on the total cost. Since $z_{\text{LB}} \leq z_{\text{opt}}$, this 5.7% is an upper bound on the true optimality gap, so the actual gap against the unknown z_{opt} is smaller. The induced access cost and migration count are not themselves lower bounds on the corresponding components of any integer-feasible schedule because the LP distributes a page fractionally across tiers; we report them as diagnostic numbers.

Table 2. Characteristics of memory tiers in simulation

	T_0	T_1	T_2	T_3
Type	L-DRAM	R-DRAM	L-PMEM	R-PMEM
Size (MB)	40	40	160	160
Load / Store (ns)	80 / 80	130 / 130	300 / 550	350 / 650

Routing every migration through T_{N-1} inflates the per-migration cost, but the inflation is dominated by the access cost savings of a beneficial migration. Therefore, the migration tendency is not distorted by the approximation. Even in the B-Tree workload, MigOpt shows a higher number of migration counts than LP.

2 Detailed Graph Construction of MigOpt

In this supplementary material, we provide a complete structure of the flow graph constructed by MigOpt. We also present a step-by-step example illustrating how MigOpt computes a near-optimal migration schedule using the Minimum Cost Maximum Flow (MCMF) formulation.

2.1 Notation and Preliminaries

Before describing the graph construction, we summarize the key notations used throughout this section in Table 3.

Table 3. Summary of notations used in graph construction

Symbol	Description
P	Set of pages managed by the system
X, Y, Z	Example pages in this supplementary material
t	Logical time units, where $0 \leq t \leq E$
a_t	Multiset of pages accessed between t and $t + 1$
T_i	i -th memory tier, where $0 \leq i \leq N - 1$
$l(p, t)$	Tier index where page p is located at time t
$R(T_i), W(T_i)$	Page read/write latency for each page of Tier- i
$A(T_i)$	Memory access latency of Tier- i
$C(T_i)$	Capacity of Tier- i
M_t	Set of pages migrated at time t
Q_p	Promotion quota per migration period
Q_d	Demotion quota per migration period

Most notations follow the definitions in §3.1 in the paper. Q_p and Q_d are newly introduced here to model migration count constraints. For the explanation, we use the example pages X, Y , and Z , where $P = \{X, Y, Z\}$. In real systems, the number of pages that can be migrated within a single migration period is limited by memory bus bandwidth and migration overhead. Q_p denotes the maximum number of pages that can be promoted (moved to faster tiers) per period, while Q_d denotes the maximum number of pages that can be demoted (moved to slower tiers) per period. These quotas are enforced through dedicated choke nodes in the flow graph (§2.2).

The baseline assumption of MigOpt is that all pages reside in the slowest tier, T_{N-1} , both before and after migration. We first solve the MCMF problem with infinite capacity for T_{N-1} , i.e., $C(T_{N-1}) = \infty$, and then correct for this assumption after obtaining the solution. The rationale for setting this capacity to infinity is to allow T_{N-1} to serve seamlessly as a logical backing store. We define the baseline state, where all pages are served from T_{N-1} , with the cost set to zero. From this baseline, positive costs are incurred for data movement

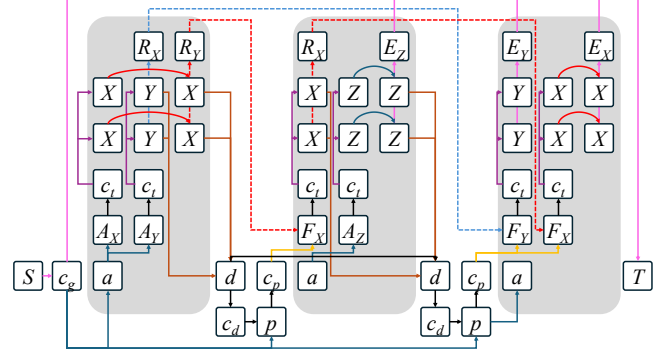


Fig. 2. Complete flow graph structure of MigOpt

(representing migration overhead), while placing pages in faster tiers yields negative costs for accesses (representing latency savings).

For example, suppose $A(T_i) = 10$ and $A(T_{N-1}) = 100$. Then, a single access incurs a cost gain (negative cost) of -90 . If multiple accesses occur, this gain accumulates. When the accumulated negative cost from repeated accesses outweighs the positive migration cost to T_i , the net cost of the migration decision becomes negative, indicating that the migration decision is more beneficial than remaining at the baseline.

2.2 Graph Construction

Fig. 2 illustrates an example of the complete graph structure for a 3-tier memory system. The gray shaded regions represent page accesses before or after each migration. For example, in the first region, pages are accessed in the order $a_t = \{X, Y, X\}$. In this graph, the horizontal axis represents the flow of time, while the vertical axis represents tiers, with higher positions corresponding to faster tiers. Our goal is to model a page's life-cycle by directing flow from source (S) to sink (K) and finding the flow configuration that minimizes total cost. Table 4 summarizes all node types and their roles in our graph formulation, and Table 5 summarizes all edge types, including the nodes they connect, their capacity constraints, associated costs, and qualitative meanings.

The key to this modeling lies in how we assign costs (or weights) to edges. Essentially, we treat the baseline—the total latency when all pages reside in T_{N-1} without any migration—as zero, and seek to determine how much latency can be reduced from this reference point. Therefore, starting from S and going directly to K represents the baseline. Flow through this edge inherently indicates residence in the lowest tier, which is why no explicit nodes for T_{N-1} appear in the graph.

To model the cost benefit gained from accesses in higher tiers, we define the *diff* metric as $\text{diff} = A(T_i) - A(T_{N-1})$. Since T_i is faster than T_{N-1} , we have $A(T_i) < A(T_{N-1})$, making *diff* always negative. The magnitude of this value represents the benefit, or time saved, per access. Because MCMF

Table 4. Node types in the MigOpt flow graph

Node type	Description and role
Source (S)	The source of the flow network, representing total memory demand. It injects one unit of flow per page and connects exclusively to capacity choke nodes. The flow exiting S is constrained by the subsequent choke node to enforce tier capacity limits.
Sink (K)	The sink that collects all terminated flows. Flow conservation requires that all flow generated at S eventually arrives at K . The baseline edge connecting directly to K allows flow to bypass all tier nodes.
Capacity choke (c_g)	The primary mechanism for enforcing hardware size limits. The edge $S \rightarrow c_g$ has capacity exactly equal to $C(T_{\text{target}})$. The meaning of "target" refers to our iterative MCMF procedure, which is discussed in detail in §2.3.
Allocation (A_X, A_Y, A_Z)	Models the initial page allocation. In MigOpt’s baseline, the default is T_{N-1} ; the A_* node allows the solver to override this. Flow traversing A_* receives a credit (negative cost) for avoiding the write to slow T_{N-1} , modeling the benefit of allocation to fast tiers. Especially, a node manages all of the allocation nodes to inject the flow.
First Access (F_X, F_Y, F_Z)	Acts as the convergence point for a page entering the active set for a migration interval. It receives flow from three origins: (1) Allocation (A_*) for newly created pages, (2) Retention from the previous interval, and (3) Promotion from pages elevated from T_{N-1} . Connects to the tier choke node c_t .
Tier choke (c_t)	A per-page, per-interval node enforcing the “One Page, One Tier” constraint with strict capacity of 1. During the Tier-0 solve, if flow passes through c_t , the node is marked as utilized. In subsequent Tier-1/2 solves, this consumed capacity prevents the page from being placed in multiple tiers simultaneously.
Page node (X, Y, Z)	The core entity representing the physical residence of the page in tier T_i . Edges connecting sequential page nodes carry access costs. From a page node, flow can diverge to: (1) Retain (R_*) to stay in tier, (2) Demote (d) to move to lower tier, or (3) End (E_*) if no longer accessed.
End (E_X, E_Y, E_Z)	Handles the exit of pages at their global last access.
Retention (R_X, R_Y, R_Z)	Models the decision to keep a page in its current tier for the subsequent interval. Flowing to R_* avoids the migration penalty. High connectivity to R_* indicates a stable hot set where pages are frequently accessed and rarely migrated.
Demotion (d)	The exhaust port for the tier, collecting pages evicted to make room for hotter pages. Flow entering d incurs the demotion cost ($R(T_i) + W(T_{N-1})$). As stated in the formulation, all remaining flow excluding R_* and E_* enters d —demotion is the fallback for pages failing to retain position due to competition.
Demotion choke (c_d)	Enforces the demotion quota Q_d , modeling bandwidth constraints. If d is saturated, the system cannot admit new pages at c_g because it cannot evict old ones fast enough.
Promotion (p)	Handles upward movement of data from T_{N-1} into target tiers. Serves flow coming from the backing store pool and routes it toward the active tier via first-access nodes.
Promotion choke (c_p)	Enforces the promotion quota Q_p , preventing the “thundering herd” problem where thousands of pages might try to migrate to T_0 simultaneously upon phase changes. The solver uses this constraint to pick only the most beneficial promotions to fill the quota.

algorithms minimize total cost, accumulating negative costs reduces the overall sum. Consequently, the solver aggressively pushes flow through edges weighted by diff, naturally prioritizing pages with high access counts for residence in faster tiers. In our graph structure, *access edges* model this behavior.

We also model the overhead incurred by migration, so demotion and promotion edges must carry migration latency as their weights. It is important to note that, unlike actual migration, our model routes a page from its current tier T_{src} down to T_{N-1} through a demotion edge, and then up to T_{dst} through a promotion edge when needed. As discussed in the paper, because the benefit from accesses dominates, this does not impose significant pressure on migration to act conservatively.

Specifically, demotion edges carry a cost of $R(T_i) + W(T_{N-1})$, while promotion edges carry a cost of $R(T_{N-1})$. The write

cost $W(T_x)$ to the destination tier is handled by placement edges, which ultimately model the positive cost associated with performing migration. As an exception, since we treat all allocations occurring in T_{N-1} as the baseline with zero cost, allocating to a different tier incurs a negative cost of $W(T_{N-1})$. Additionally, to model capacity constraints and exclusive placement (i.e., ensuring that the same page does not reside in different tiers at the same time) in our flow graph, we introduce choke edges with limited flow capacity and choke nodes that connect them.

2.3 Iterative MCMF Procedure

MigOpt does not solve for all tiers simultaneously. Instead, it employs an iterative, greedy-by-tier approach that ensures the fastest memory tiers are populated with pages yielding the highest global benefit. MCMF procedure iterates as follows:

Table 5. Complete specification of edges in the MigOpt flow graph

Edge Type	Edge	Capacity	Cost	Physical Interpretation
Capacity choke edge	$S \rightarrow c_g$	$C(T_i)$	0	Constraint for tier capacity
Baseline edge	$c_g \rightarrow K$	∞	0	Default path: remain in T_{N-1}
Allocation edge	$a \rightarrow A_*$	∞	$-W(T_{N-1})$	Allocation credit: avoid default T_{N-1} write
Tier choke edge	F_* or $A_* \rightarrow c_t$	1	0	Exclusive placement enforcement
Initial placement edge	$c_t \rightarrow X$	1	$W(T_i) + \text{diff}$	Net allocation: write to T_i + first access benefit
Access edge (intra-period)	$X \rightarrow X$	1	diff	Access benefit: latency savings in T_i vs. T_{N-1}
Retention edge (inter-period)	$X \rightarrow R_X \rightarrow X$	1	0	No migration cost for tier persistence
Demotion edge	$X \rightarrow d$	1	$R(T_i) + W(T_{N-1})$	Migration penalty: read from T_i , write to T_{N-1}
Demotion quota edge	$d \rightarrow c_d$	Q_d	0	Constraint for demotions
Promotion edge	$c_d/c_g \rightarrow p$	∞	$R(T_{N-1}) + W(T_i)$	Migration cost: read from T_{N-1} , write to T_i
Promotion quota edge	$p \rightarrow c_p$	Q_p	0	Constraint for promotions
End edge	$X \rightarrow E_X \rightarrow K$	∞	0	Termination

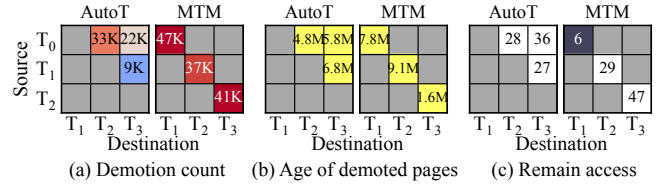
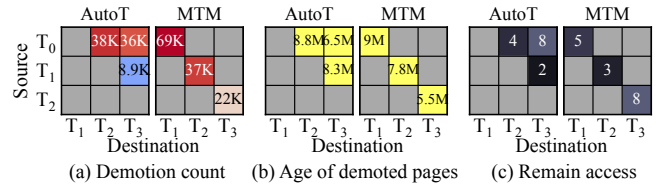
Step 1: Tier-0 Optimization. The algorithm constructs the graph with T_0 as the target, setting the source flow capacity to $C(T_0)$. All diff costs are calculated as $A(T_0) - A(T_{N-1})$, representing the maximum possible latency reduction. The MCMF solver identifies pages that (1) are accessed frequently enough to justify allocation/migration costs and (2) fit within the limited T_0 capacity. Pages not selected remain on the baseline path.

Step 2: Pruning and State Retention. Once optimal pages for T_0 are identified, these decisions are fixed, and the tier choke nodes (c_t) for selected pages are marked as occupied. A page assigned to T_0 at time t is effectively removed from the candidate pool for T_1 at time t , strictly enforcing the constraint that a page can reside in only one tier at any given time.

Step 3: Tier-1 Optimization. The graph is reconstructed with T_1 as the target. The source capacity is updated to $C(T_1)$, and costs are recalculated as $A(T_1) - A(T_{N-1})$. Since Remote DRAM is slower than Local DRAM, the benefits are smaller, resulting in lower migration pressure. The solver identifies the next-best pages—those that did not qualify for T_0 but are sufficiently hot to justify Remote DRAM over PMEM.

Step 4 and Beyond: Remaining Tiers and Default. The process repeats for T_2 and subsequent tiers. Any page flow not assigned to Tiers 0, 1, or 2 constitutes the residual set, which is implicitly assigned to T_{N-1} . Since T_{N-1} serves as the baseline, the cost of residing there is normalized to zero.

This iterative process is the key mechanism for enforcing exclusive placement. The critical constraint across MCMF iterations is the $F \rightarrow c_t$ edge with capacity 1. Once this edge is saturated for a particular page-time combination in an earlier iteration, it cannot carry additional flow in subsequent iterations, thereby preventing the same page from being placed in multiple tiers simultaneously.


Fig. 3. Demotion results of AutoT and MTM on B-Tree

Fig. 4. Demotion results of AutoT and MTM on XSBench

3 Demotion Behavior of Existing Techniques

Fig. 3 and Fig. 4 present the demotion behaviors of AutoT and MTM on B-Tree and XSBench, respectively. Both workloads exhibit trends similar to those observed in YCSB-D.

In B-Tree, demoted pages under both AutoT and MTM are spread across all available demotion paths (Fig. 3(a)). As in YCSB-D, these pages tend to have old ages (Fig. 3(b)) and continue to receive non-trivial numbers of accesses even after being placed in lower tiers (Fig. 3(c)). We also observed that a similar pattern appears in XSBench (Fig. 4).

In summary, across all workloads, both AutoT and MTM exhibit demotion behaviors fundamentally different from those of MigOpt, leading to inefficient tier utilization and degraded performance.

4 Constructing Histogram

To build the histogram of page accesses, MigFlow first inserts the information of the pages that enter the access zone into the histogram's leftmost bin. When MigFlow detects an access to a page in this zone, MigFlow updates its access count in the per-page metadata, and if the access count exceeds its bin's boundary, MigFlow moves its location of the histogram from the original bin to the next (right) bin. Furthermore, to preserve temporal sensitivity, we apply an exponential moving average (EMA) that halves the accumulated access count once every 2M page samples. This decay is applied to

all pages within each bin by updating the access count in the per-page metadata through the stored pointer, and the bin's entries are shifted left accordingly.

References

- [1] Q. Huangfu and J. A. J. Hall. 2018. Parallelizing the Dual Revised Simplex Method. *Mathematical Programming Computation* 10, 1 (2018), 119–142.
- [2] Anil P. Kamath, Narendra Karmarkar, K. G. Ramakrishnan, and Mauricio G. C. Resende. 1990. Computational Experience with an Interior Point Algorithm on the Satisfiability Problem. *Annals of Operations Research* 25, 1 (1990), 43–58.